European Commission

DPP Registry API Integration Guide for Economic Operators

DG Grow Digital Solutions

Last published 2026-10-30 v1.0



Table of Contents

Registry registration and activity tracking 4

Chapter 1. Introduction 5
- 1.1 About this Guide 5
  - Purpose and audience 5
  - DPP Registration and activity tracking at a glance 5
- 1.2 Core concepts 5
  - Submission acceptance 5
  - Task lifecycle and business outcome 5
  - The unique product identifier 6
  - Digital product passport validation 6
  - Organisation scoping and the verification prerequisite 6

Chapter 2. Getting started 7
- 2.1 Prerequisites and setup 7
  - Authentication 7
  - Environments and base URLs 7
  - Submission headers 7
  - Fetch reference data first 7
  - First end-to-end registration flow 7

Chapter 3. Registering DPPs 9
- 3.1 Registration submission 9
  - The submission contract 9
  - Idempotent submission 9
  - Reading the 202 9
- 3.2 Single and multiple-item submissions 10
  - Registration atomicity 10
- 3.3 Tracking processing 10
- 3.4 Validation and errors 11
  - HTTP status semantics (submission) 11

Chapter 4. Tracking DPP Registration Requests 12
- 4.1 Listing tasks 12
  - Endpoint 12
  - Filtering 12
  - Pagination and sorting 12
- 4.2 Viewing task detail and outcome 12
  - The task detail envelope 12
  - Task-specific data 13
- 4.3 Per-item processing evidence 14
  - Endpoint 14
  - JSON and CSV representations 14
  - Task-specific item data 14

Chapter 5. Putting it together 16
- 5.1 End-to-end workflows 16
  - Registering a single DPP 16
  - Registering multiple DPPs and monitor 16
  - Correlating registration and activity tracking 16

5.2 Cross-cutting concerns 16
Correlation and traceability 16
Retries and idempotency 17
Limits and performance 17
Security and data handling 17

Chapter 6. Reference 18
6.1 Endpoint index 18

Glossary of Terms 19

Registry registration and activity tracking

Functionality	Specification	Version
Registry	openapi-dpp-registry-public.yml	1.0.0
Activity Dashboard	openapi-dpp-dashboard-public.yml	1.0.0
Shared components	openapi-dpp-common-public.yml	1.0.0

This guide explains how an Economic Operator (EO) requests to register Digital Product Passports (DPPs) and follows the outcome of each registration request. The Registry receives, validates and processes DPP registration requests. The Activity Dashboard provides visibility into asynchronous processing activities, allowing users to monitor task status, review results and export available outcomes in CSV format.

> **NOTE**
> In this guide, DPP refers to the Digital Product Passport artefact or data made available by the Economic Operator. Registry refers to the platform used to register DPPs.

Chapter 1. Introduction

1.1 About this Guide

Purpose and audience

This guide is intended for Economic Operators and the developers or architects integrating their systems with Registry and Activity Dashboard APIs. It assumes familiarity with HTTP, JSON and Bearer authentication. It explains the registration process, the API calls involved and the client behaviour required to complete and monitor a registration request.

DPP Registration and activity tracking at a glance

Registry accepts DPP registration submissions containing one or more DPP items. Each submission contains a common product classification and between 1 and 100 **submissionItems**.

Activity Dashboard provides centralised visibility into the asynchronous task created for each accepted submission, including its lifecycle status, business outcome and per-item processing evidence.

1.2 Core concepts

Submission acceptance

When Registry accepts a submission, it returns 202 Accepted and opens an asynchronous task used to process and track the registration:

1. The response includes a **taskId**, which is the identifier used for subsequent Activity Dashboard operations.
2. When **estimatedPeriodCompletionTime** is provided, use that ISO 8601 duration as guidance for when to query Activity Dashboard for the first time. The estimate is not a completion guarantee.
3. The acceptance response does not represent the current task lifecycle status. Retrieve **GET /api/v1/tasks/{taskId}** to determine whether the task is **PENDING**, **PROCESSING**, **COMPLETED** or **ERROR**.

Task lifecycle and business outcome

Activity Dashboard separates the lifecycle of the asynchronous task from the business outcome produced by that task.

- **PENDING:** means that the task has been created and is waiting to be processed.
- **PROCESSING:** means that asynchronous processing is in progress.
- **COMPLETED:** means that task processing finished and a business outcome is available.
- **ERROR:** means that the task itself could not be complete successfully.

For a **COMPLETED** task, inspect **outcome.outcomeResult** to determine the business outcome:

- **SUCCEEDED:** positive business outcome.
- **FAILED:** negative business outcome, for example when one or more submitted DPP items are non-

compliant. Registration submissions are atomic: if any submitted item is non-compliant, none of the items in that submission are registered.

A business outcome of **FAILED** does not mean that the asynchronous task failed technically. A task may therefore have **status=COMPLETED** and **outcomeResult=FAILED**.

The unique product identifier

The **uniqueProductIdentifier (UPI)** shall be in a URL format. Registry uses the UPI exactly as submitted as the initial URL of the DPP resource.

Digital product passport validation

Upon submission for registration, the registry shall read the digital product passport content and validate it in accordance with Commission implementing regulation (EU) 2026/1778. The DPP resource must therefore be reachable at validation time. An unreachable resource results in UPI_NOT_RESOLVABLE. **Registry does not persist the fetched DPP payload content**.

Organisation scoping and the verification prerequisite

Registry operations are scoped to the Organisation associated with the authenticated principal, clients do not pass an organisation ID in registration requests. An Economic Operator must complete Organisation verification during onboarding before using Registry integration capabilities. Requests from an Organisation that is not eligible for registration are rejected.

Chapter 2. Getting started

2.1 Prerequisites and setup

Authentication

Protected endpoints require a **Bearer JWT in Authorization: Bearer <token>**. For machine-to-machine integrations using Service Accounts, see the ***DPP Registry Service Account Technical Integration Guide*** for the authentication and access-token acquisition procedure.

The Organisation and authenticated principal are derived from the token.

Environments and base URLs

Use the Registry and Activity Dashboard base URLs published for the target environment. Do not derive deployment endpoints from placeholder or local-development host values in the API specifications.

Submission headers

Header	Required	Purpose
X-Correlation-Id	No	Correlates the request with downstream processing and support diagnostics. If omitted, the backend generates and propagates one.
Idempotency-Key	Yes	Provides replay-safe deduplication (maximum 128 characters). Reusing the same key returns the original accepted task.

Fetch reference data first

Before submitting a registration, call **GET /api/v1/dpp/product-groups** and use the **productGroup** and, where applicable, **productSubGroup** corresponding to the product being registered. The returned values are Registry product-classification reference data and are not arbitrary selections.

First end-to-end registration flow

- **Step 1:** Confirm that the Organisation has completed verification and that the caller is authorised to register DPPs. For a Service Account, use a Service Account configured with the Register DPPs role.
- **Step 2:** Retrieve the Registry product-classification reference data with **GET /api/v1/dpp/product-groups** and use the **productGroup** and, where applicable, **productSubGroup** corresponding to the DPP being registered.
- **Step 3:** Ensure that each DPP is reachable through its **uniqueProductIdentifier**, then send POST **/api/v1/dpp/submission**. For a first single-DPP registration, provide exactly one element in **submissionItems**. Store the returned **taskId**. If **estimatedPeriodCompletionTime** is returned, use it as guidance for when to query the task for the first time.
- **Step 4:** After the estimated period, call **GET /api/v1/tasks/{taskId}** in Activity Dashboard. If the task is **PENDING** or **PROCESSING**, continue querying until it reaches a terminal state. If it is

COMPLETED, inspect outcome and **specificTaskData**. If it is **ERROR**, treat the task as a technical processing failure. Use **GET /api/v1/tasks/{taskId}/items** when per-item processing evidence is required.

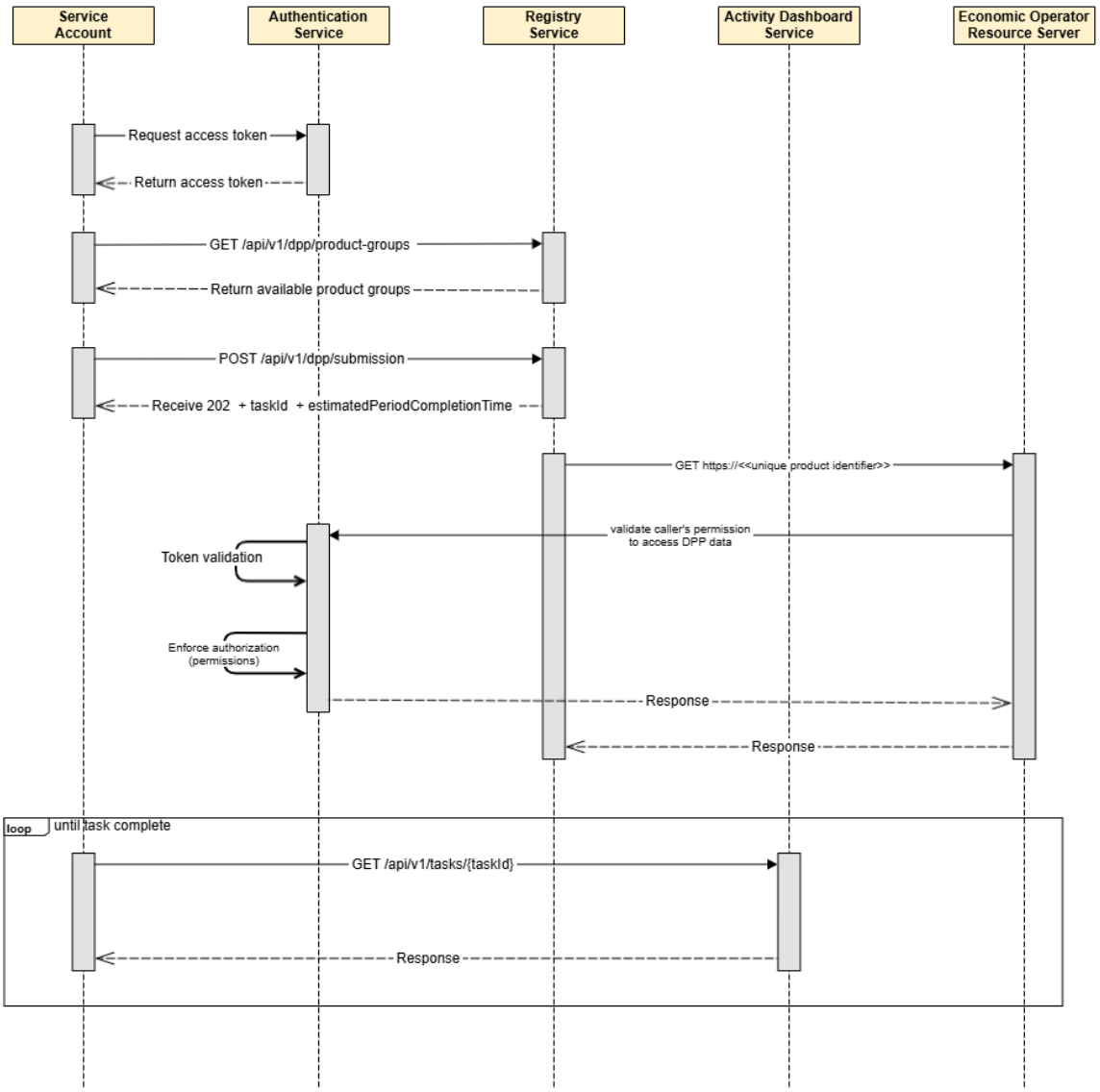


Figure 1 - End-to-end registration flow

Chapter 3. Registering DPPs

3.1 Registration submission

The submission contract

Send **POST /api/v1/dpp/submission** with Content-Type: application/json:

```
{
  "productGroup": "BATTERIES",
  "productSubGroup": "EV",
  "submissionItems": [
    {
      "uniqueProductIdentifier": "https://example.com/dpp/battery/0001",
      "granularity": "item",
      "batchIdentifier": "xxx-000",
      "modelIdentifier": "yyy-111"
    }
  ]
}
```

uniqueProductIdentifier is required for every submitted item. The example above also includes fields applicable to this **productGroup**. Other product groups may define additional or different product-specific fields. Always use the fields defined by the Registry contract for the selected product group and product sub-group.

A registration submission always uses the same JSON contract. Use one **submissionItems** element to register a single DPP and multiple elements to register several DPPs sharing the same request-level product classification.

Idempotent submission

Send **Idempotency-Key** with every registration submission. The key identifies one logical action within the REST resource to which the request is submitted.

When retrying the same logical submission, reuse the same Idempotency-Key. Registry associates the retry with the original action and returns the outcome associated with that action rather than creating a duplicate submission.

A new or modified logical submission to the same resource requires a new Idempotency-Key. Reusing a key already associated with another logical action on the same resource is treated as an idempotency conflict and does not create a new action.

Idempotency keys are scoped to the REST resource. The same key value may therefore be used independently for requests to different resources without creating an idempotency conflict.

Clients should treat an Idempotency-Key as bound to a single logical action within a given REST resource and should not reuse the same key for a different action on that resource.

Reading the 202

A 202 Accepted response confirms that Registry accepted the request for asynchronous processing, **it does not mean that registration is complete.**

Field	Meaning
taskId	Primary identifier used to track the asynchronous operation through Activity Dashboard.
taskType	Type of asynchronous task created by the submission. For DPP registration: DPP_REGISTRATION.
submittedAt	Time at which the task-bearing request was accepted.
correlationId	End-to-end correlation identifier for tracing and support.
estimatedPeriodCompletionTime	Optional estimated time until completion, expressed as an ISO 8601 duration, for example PT10S. Use it as guidance before the first task query.
updatedAt	Timestamp of the latest task update returned with the acknowledgement.

3.2 Single and multiple-item submissions

The same **POST /api/v1/dpp/submission** operation and request schema are used for both single and multiple-item registrations.

To register a single DPP, provide one element in **submissionItems**. To register multiple DPPs, provide between 2 and 100 elements.

productGroup and, when applicable, **productSubGroup** apply to the complete submission. DPPs that belong to different product classifications must therefore be submitted in separate requests.

Each **submissionItems** element requires its own **uniqueProductIdentifier** and may contain additional fields defined by the registration schema applicable to the selected product group.

A registration submission is atomic. Registry validates all submitted items before completing the registration. If every submitted item is compliant, the submission can complete successfully and the DPPs are registered.

Registration atomicity

If one or more submitted items are non-compliant, the complete submission has a negative business outcome and **none of the submitted DPPs are registered**, including items that individually passed validation.

The task result reports the number of submitted, compliant and non-compliant items. Use the per-item processing evidence to identify which items caused the submission to fail.

3.3 Tracking processing

After a registration submission is accepted, use the returned **taskId** to track it through Activity Dashboard.

When **estimatedPeriodCompletionTime** is provided in the 202 Accepted response, wait approximately the indicated period before making the first **GET /api/v1/tasks/{taskId}** request.

If the task is still **PENDING** or **PROCESSING**, the client should apply a Fibonacci polling backoff for subsequent requests. Recommended polling intervals are 1, 1, 2, 3, 5, 8, 13, 21, 34 and 55 seconds, after which the client shall cap its own polling interval at 60 seconds until the task reaches a terminal state.

If an HTTP response includes Retry-After, wait at least the period indicated by that header before sending the next request. Retry-After takes precedence over the client-generated polling interval and may therefore exceed the normal 60-second polling cap.

When the task reaches **COMPLETED**, inspect outcome for the overall business result and **specificTaskData.processingInfo** for the aggregate compliance result of the submitted items. Use **GET /api/v1/tasks/{taskId}/items** when per-item processing evidence is required.

If the task reaches **ERROR**, treat it as a technical processing failure.

See **Section 1.2 Core concepts** for the distinction between task lifecycle status and business outcome.

3.4 Validation and errors

HTTP status semantics (submission)

Code	Meaning	EO action
202	**Accepted for asynchronous processing only.** The request has been accepted for processing, this does not mean that the submitted DPPs have been accepted or registered.	Monitor the task and retrieve the result.
400	The request envelope is invalid.	Correct the request structure and submit again with a new Idempotency-Key.
403	The Organisation is not eligible for registration, or the caller lacks the required permission.	Confirm Organisation eligibility and caller permissions.
409	The request is duplicate or conflicts with the current resource state or an idempotency rule.	Review the response and the current state. Use a new Idempotency-Key only when submitting a new or modified logical request.
413	The request body exceeds the maximum allowed size of 10 MB.	Reduce the request size and submit the modified request using a new Idempotency-Key.
422	The request is syntactically valid but cannot be processed because semantic validation is unavailable or the submitted data is logically invalid.	Inspect the returned error details. Correct the request and use a new Idempotency-Key when the data is invalid. Retry the same logical request with the same key when the response indicates temporary validation unavailability.
429	Rate limit exceeded.	Wait at least the period indicated by Retry-After before retrying. For a registration submission, retry the same logical request with the same Idempotency-Key.
500	A technical error occurred.	Retry the same logical submission with the same Idempotency-Key.

Chapter 4. Tracking DPP Registration Requests

4.1 Listing tasks

Endpoint

GET /api/v1/tasks returns a paginated **PageActivityDashboardRow** view of asynchronous tasks visible to the caller's Organisation. For Registry integration, use **taskType=DPP_REGISTRATION** to focus on DPP registration tasks.

The response uses a task summary containing:

- taskId
- taskType
- correlationId
- status
- actor
- organisationId
- submittedAt
- completedAt
- outcome, only if **extended parameter is true**.

Filtering

Parameter	Values
status	PENDING, PROCESSING, COMPLETED, ERROR
taskType	For Registry integration, use DPP_REGISTRATION. The API also supports task types for other platform processes.
extended	By default, task-list rows contain only the common task summary. Use **extended=true** when the terminal outcome is also required in each returned row.
submittedFrom, submittedTo	Inclusive ISO-8601 date-time bounds applied to submittedAt

Pagination and sorting

Use page for the zero-based page number (default 0), size for the page size (default 25, maximum 100), and sort for ordering, for example **submittedAt,desc**. The default order is **submittedAt,desc**. Supported sort fields are **submittedAt**, **status**, **completedAt** and **taskType**.

4.2 Viewing task detail and outcome

The task detail envelope

Use **GET /api/v1/tasks/{taskId}** to retrieve the current task detail.

The response uses a common task envelope containing:

- **taskId**
- **taskType**
- **correlationId**
- **status**
- **actor**
- **organisationId**
- **submittedAt**
- **completedAt**
- **outcome**, when a business outcome is available
- **specificTaskData**, containing data specific to the task type

taskType determines the schema of **specificTaskData**. The same task detail endpoint is therefore used for DPP registration and other asynchronous processes exposed through Activity Dashboard.

outcome contains the common business result through **outcomeResult** and **outcomeMessage**. **specificTaskData** contains the domain-specific data produced by the task.

Task-specific data

taskType	specificTaskData	Key fields
DPP_REGISTRATION	DppRegistrationOutcomeData	productGroup, productSubGroup, submissionRequestId, processingInfo.submittedItems, processingInfo.compliantItems, processingInfo.nonCompliantItems
ORGANISATION_VERIFICATION_DECLARATION_GENERATION	OrganisationVerificationDeclarationGenerationOutcomeData	enrolmentApplicationId
ORGANISATION_VERIFICATION_SUBMISSION	OrganisationVerificationSubmissionOutcomeData	enrolmentApplicationId, validFrom, validUntil, signedBy

For a DPP registration task, processingInfo provides aggregate compliance counters for the submitted items: submittedItems is the total number of items included in the submission, compliantItems is the number of items that individually passed validation and nonCompliantItems is the number of items that did not pass validation.

These counters do **not** represent partial registration. Registration is atomic: if nonCompliantItems is greater than zero, none of the submitted DPPs are registered. Retrieve GET /api/v1/tasks/{taskId}/items when item-level evidence is required.


```
{
  "taskId": "task_01J1REG001",
  "taskType": "DPP_REGISTRATION",
  "correlationId": "corr_01J1REG",
  "status": "COMPLETED",
  "organisationId": "org_01H8XGP",
  "submittedAt": "2026-10-05T11:11:21.718Z",
  "completedAt": "2026-10-05T11:12:03.004Z",
  "outcome": {
    "outcomeResult": "FAILED",
    "outcomeMessage": {
      "code": "REJECTED",
      "message": "DPP submission contains non-compliant items"
    }
```

```
  },
  "specificTaskData": {
    "productGroup": "BATTERIES",
    "productSubGroup": "EV",
    "submissionRequestId": "reg_01J1REG001",
    "processingInfo": {
      "submittedItems": 50,
      "compliantItems": 49,
      "nonCompliantItems": 1
    }
  }
}
```

DPP Registration Task Example

4.3 Per-item processing evidence

Endpoint

Use **GET /api/v1/tasks/{taskId}/items** to retrieve paginated per-item processing evidence for a task.

By default, the endpoint returns application/json. Each item contains common processing information together with task-specific item data determined by **taskType**.

Use the **page** and **size** query parameters to navigate the result set.

JSON and CSV representations

The same endpoint also supports **text/csv.**
When CSV is requested, the API returns the same requested page rendered as CSV, with one row per processed item. The response may include a Content-Disposition header with a suggested attachment filename.
CSV is an alternative representation of the paginated item result, it is not a separate full-result export operation. The page and size parameters apply equally to JSON and CSV responses.

Task-specific item data

taskType	specificTaskItemData	Key fields
DPP_REGISTRATION	DppRegistrationItemOutcomeData	uniqueProductIdentifier, uniqueRegistryIdentifier, productGroup, productSubGroup, submissionRequestId, granularityLevel, batchIdentifier, modelIdentifier
ORGANISATION_VERIFICATION_DECLARATION_GENERATION	OrganisationVerificationDeclarationGenerationItemOutcomeData	enrolmentApplicationId
ORGANISATION_VERIFICATION_SUBMISSION	OrganisationVerificationSubmissionItemOutcomeData	enrolmentApplicationId

outcomeMessage provides the primary item-level result or error message. **outcomeDetails** contains any additional granular messages associated with that item. **specificTaskItemData** contains business identifiers and other task-specific information.

```
{
  "taskId": "task_01J1REG001",
  "taskItemIndex": 1,
  "taskType": "DPP_REGISTRATION",
  "outcomeMessage": {
```

```
    "code": "ITEM_NON_COMPLIANT",
    "message": "UPI not resolvable"
  },
  "outcomeDetails": [
    {
      "code": "UPI_NOT_RESOLVABLE",
      "message": "The uniqueProductIdentifier could not be resolved over HTTPS"
    }
  ],
  "specificTaskItemData": {
    "uniqueProductIdentifier": "https://example.com/dpp/battery/0002",
    "productGroup": "BATTERIES",
    "productSubGroup": "EV",
    "submissionRequestId": "reg_01J1REG001",
    "granularityLevel": "item"
  }
}
```

DPP Registration Task Item Example

Chapter 5. Putting it together

5.1 End-to-end workflows

Registering a single DPP

- **Step 1:** Confirm that the Organisation has completed verification and that the caller is authorised to register DPPs.
- **Step 2:** Retrieve GET /api/v1/dpp/product-groups and use the product classification corresponding to the DPP.
- **Step 3:** Submit POST /api/v1/dpp/submission with one submissionItems element. Store the returned taskId and, when provided, estimatedPeriodCompletionTime.
- **Step 4:** After the estimated period, retrieve GET /api/v1/tasks/{taskId}. If the task is PENDING or PROCESSING, continue querying it until it reaches COMPLETED or ERROR.
- **Step 5:** For a COMPLETED task, inspect outcome and specificTaskData. Retrieve /api/v1/tasks/{taskId}/items if per-item evidence is required.

Registering multiple DPPs and monitor

- **Step 1:** Submit the same POST /api/v1/dpp/submission request with multiple submissionItems entries, up to the contract limit of 100.
- **Step 2:** Track the returned taskId through GET /api/v1/tasks/{taskId}.
- **Step 3:** When the task is COMPLETED, inspect outcome and specificTaskData.processingInfo. submittedItems, compliantItems and nonCompliantItems provide the aggregate compliance result of the submission.
- **Step 4:** If nonCompliantItems is greater than zero, none of the submitted DPPs have been registered. Retrieve GET /api/v1/tasks/{taskId}/items to identify the non-compliant items, correct the submission and submit the complete set again as a new logical registration request using a new Idempotency-Key.

Correlating registration and activity tracking

The **taskId** returned in the 202 response links the Registry submission to Activity Dashboard task detail and per-item processing operations. Use the identifiers as follows:

- **taskId** - identifies the asynchronous task and is used for status and result operations.
- **correlationId** - supports end-to-end tracing and should be included when raising a support request.

5.2 Cross-cutting concerns

Correlation and traceability

Send **X-Correlation-Id** when you need to correlate client-side logs with Registry processing. If omitted, the backend generates and propagates a value. The identifier is returned as **correlationId** on task and detail responses and as **traceId** on error responses.

Retries and idempotency

When retrying the same logical submission, reuse the original **Idempotency-Key**. Registry returns the original accepted task for the same key. Use a new key only for a new logical submission.

For Activity Dashboard polling, follow the polling and Retry-After rules defined in **Section 3.3**.

Limits and performance

A registration submission contains 1–100 **submissionItems**. Treat **estimatedPeriodCompletionTime** as guidance only. Use task status to determine completion.

Security and data handling

API visibility is scoped to the Organisation associated with the authenticated principal. Registry does not persist the DPP payload content fetched while resolving a UPI.

Chapter 6. Reference

6.1 Endpoint index

Method & path	Service	Purpose
GET /api/v1/dpp/product-groups	Registry	Retrieve valid product-group values
POST /api/v1/dpp/submission	Registry	Submit one or more DPP registration items for asynchronous processing.
GET /api/v1/tasks	Activity Dashboard	List tasks visible to the caller's Organisation, with filtering and pagination.
GET /api/v1/tasks/{taskId}	Activity Dashboard	Retrieve task lifecycle status, terminal business outcome and task-specific data.
GET /api/v1/tasks/{taskId}/items	Activity Dashboard	Retrieve paginated per-item processing evidence as JSON or CSV.

Glossary of Terms

Term	Meaning
Activity Dashboard functionality	Provides visibility into asynchronous tasks, their lifecycle and business outcome, and paginated per-item processing evidence available as JSON or CSV.
Digital Product Passport (DPP)	Digital Product Passport artefact or data made available by the Economic Operator and referenced by its uniqueProductIdentifier.
Economic Operator (EO)	Organisation that registers DPPs and integrates its systems with Registry APIs.
Idempotency-Key	Required submission header used for replay-safe deduplication. Reusing a key returns the original accepted task.
Registry	Platform functionality used to submit, validate and register DPPs.
Task	Asynchronous processing record exposed through Activity Dashboard and identified by taskId.
taskId	Primary identifier returned when an asynchronous operation is accepted and used to retrieve task detail and per-item processing evidence through Activity Dashboard.
uniqueProductIdentifier (UPI)	URL that identifies the DPP resource. Registry uses it as submitted when resolving and validating the referenced resource.
X-Correlation-Id	Optional request header used for request correlation and distributed tracing. If omitted, the backend generates and propagates one.